

WishHive — Complete Dart/Flutter Guide

Every Dart concept used in this app, explained with real code examples. Interview-ready reference.



Table of Contents

1. Dart Language Fundamentals
2. Object-Oriented Programming (OOP)
3. Null Safety
4. Collections & Functional Operations
5. Async Programming (Future, Stream, async/await)
6. Design Patterns Used
7. Flutter Widget System
8. State Management (Riverpod)
9. Firebase Integration
10. App Architecture & Project Structure
11. Complete Feature List
12. Future Improvements

1. Dart Language Fundamentals

1.1 Variables & Data Types

Dart is strongly typed. We use `final` for values that never change after assignment, and `const` for compile-time constants.

```
// From app_theme.dart - compile-time constants
static const Color primaryAmber = Color(0xFFF5A623);
static const Color backgroundLight = Color(0xFFFFF7);

// From upload_service.dart - variables
bool _isUploading = false;    // Mutable variable
final task = state.first;     // Assigned once, never changed
var fileToUpload = file;      // Type inferred as File
```

Interview Tip

`const` = compile-time constant (must be known before the app runs).

`final` = runtime constant (set once, can be computed at runtime).

`var` = type inferred, mutable (use when value changes).

1.2 String Interpolation

Dart uses `$variable` or `${expression}` inside strings.

```
// From contacts_page.dart
final inviteText = 'Hey $name! 🎉 Join me on WishHive';

// From image_storage_service.dart
final savedPath = '${imageDir.path}/${fileName}';

// From upload_service.dart - complex expression
debugPrint('[UploadService] Compressed: ${file.lengthSync()} → ${fileToUpload.l
```

1.3 Enums

Enums define a fixed set of options. Used in `HiveModel` for privacy levels.

```
// From hive_model.dart
enum HivePrivacy { public, private, friends, specific }
```

```
// Usage – converting enum to/from String for Firestore
'privacy': privacy.name, // Enum → String: "public"

// Parsing back from Firestore
static HivePrivacy _parsePrivacy(String? value) {
  switch (value) {
    case 'public': return HivePrivacy.public;
    case 'friends': return HivePrivacy.friends;
    default: return HivePrivacy.private;
  }
}
```

1.4 Functions — Arrow Syntax & Named Parameters

```
// Arrow syntax – single expression functions
Set<String> build() ⇒ {};
static bool isAssetPath(String path) ⇒ path.startsWith('assets/');

// Named parameters with required keyword
const HiveModel({
  required this.id,           // Must be provided
  required this.title,
  this.imageUrl = '',        // Optional with default
  this.privacy = HivePrivacy.private,
});

// Calling with named parameters
await toggleWishFulfillment(
  hiveOwnerId: uid,
  wishId: wish.id,
  fulfillerId: uid,
  fulfillerName: 'A Friend',
);
```

2. Object-Oriented Programming (OOP)

2.1 Classes & Constructors

```
// From user_model.dart — Regular class with const constructor
class UserModel {
  final String uid;
  final String email;
  final String displayName;
  final String? username;           // Nullable field
  final List<FriendProfile> friends; // Generic list

  const UserModel({                // const constructor
    required this.uid,
    required this.email,
    this.friends = const [],       // Default empty list
  });
}
```

2.2 Factory Constructors

Factory constructors can return existing instances or transform data. We use them everywhere to parse Firestore documents.

```
// From wish_model.dart — Factory that parses Firestore data
factory WishModel.fromFirestore(DocumentSnapshot doc) {
  final data = doc.data() as Map<String, dynamic>? ?? {};
  return WishModel(
    id: doc.id,
    name: data['name'] as String? ?? 'No Name',
    cost: (data['cost'] as num?)?.toDouble() ?? 0.0,
    createdAt: (data['createdAt'] as Timestamp?)?.toDate(),
    ownerSeen: data['ownerSeen'] as bool? ?? true,
  );
}
```

Interview Tip — Why Factory Constructors?

Regular constructors ALWAYS create a new instance. Factory constructors can:

1. Parse/transform data before creating the object
2. Return a cached instance (singleton pattern)

3. Return a subtype

In our app, `fromFirestore` factory constructors handle type casting, null safety defaults, and data transformation.

2.3 Inheritance & Mixins

```
// Extending a class – StatefulWidget pattern
class ContactsPage extends ConsumerStatefulWidget { ... }

// Mixin – adds animation capability without full inheritance
class _ContactsPageState extends ConsumerState<ContactsPage>
    with SingleTickerProviderStateMixin { ... }

// UploadService extends Notifier (Riverpod state management)
class UploadService extends Notifier<List<UploadTask>> { ... }
```

extends vs with vs implements

Keyword	Purpose	Example in WishHive
<code>extends</code>	Inherit ONE parent class (gets all methods)	<code>ConsumerStatefulWidget</code>
<code>with</code>	Add behavior from mixins (multiple allowed)	<code>SingleTickerProviderStateMixin</code>
<code>implements</code>	Promise to implement an interface	Not used in our app

2.4 Static Methods

```
// From image_storage_service.dart – Static = belongs to the CLASS, not instance
class ImageStorageService {
    static Future<String> saveImage(File imageFile) async { ... }
    static bool isLocalPath(String path) { ... }
```

```
static bool isNetworkPath(String path) { ... }
}

// Called without creating an instance:
ImageStorageService.isLocalPath(path); // Not: ImageStorageService().isLocalPa
```

3. Null Safety

Dart's null safety prevents null reference errors at compile time. Every variable is non-null by default.

3.1 Nullable Types (?)

```
// From user_model.dart
final String? username; // Can be null
final String? photoUrl; // Can be null
final String email; // CANNOT be null

// From wish_model.dart
final DateTime? date; // Optional date
final DateTime? createdAt; // Optional timestamp
```

3.2 Null-Aware Operators

```
// ?? – Null coalescing (provide default if null)
data['name'] as String? ?? 'No Name'
user?.friendRequestsReceived.length ?? 0

// ?. – Null-safe member access
theme.textTheme.bodyLarge?.copyWith(color: Colors.grey)

// ?. with method calls
(data['cost'] as num?).toDouble() ?? 0.0 // Cast, then safe call

// ! – Force unwrap (use ONLY when you're 100% sure it's not null)
final uid = _uid;
```

```
if (uid == null) throw Exception('Not authenticated');  
// After the check, uid is guaranteed non-null
```

3.3 Type Casting (**as**)

```
// From user_model.dart – Firestore returns dynamic types  
final data = doc.data() as Map<String, dynamic>? ?? {};  
  
// Safe type casting chain:  
(data['quantity'] as num?).toInt() ?? 1  
// ↑ cast to num?   ↑ safe call   ↑ default
```

4. Collections & Functional Operations

4.1 Lists

```
// From user_model.dart – Immutable default list  
this.friends = const []  
  
// Creating list from Firestore dynamic data  
List<String>.from(data['friendRequestsSent'] ?? [])  
  
// Spread operator ( ... ) – merge lists  
state = [ ... state, task];    // Add item to immutable list  
state = { ... state, id};      // Add item to immutable set
```

4.2 Map / Where / Any / toSet

```
// .map() – Transform each element  
friends.map((f) ⇒ f.toMap()).toList()  
  
// .where() – Filter elements  
state.where((t) ⇒ t ≠ task).toList()  
  
// .any() – Check if ANY element matches
```

```
me.friends.any((f) ⇒ f.uid == user.uid)

// .toSet() – Remove duplicates
final friendEmails = me.friends.map((f) ⇒ f.email.toLowerCase()).toSet();

// .intersection() – Find common elements between two sets
contactEmails.intersection(friendEmails).isEmpty
```

4.3 Maps (Dictionaries)

```
// From hive_model.dart – Converting model to Firestore map
Map<String, dynamic> toFirestore() {
  return {
    'title': title,
    'imageUrl': imageUrl,
    'privacy': privacy.name,          // Enum to String
    'itemCount': itemCount,
    'createdAt': FieldValue.serverTimestamp(),
  };
}
```

5. Async Programming

5.1 Future & async/await

A **Future** represents a value that will be available later. **async/await** makes async code look synchronous.

```
// From image_storage_service.dart
static Future<String> saveImage(File imageFile) async {
  final appDir = await getApplicationDocumentsDirectory();
  // ↑ Pauses here until the directory is fetched
  final imageDir = Directory('${appDir.path}/beehive_images');

  if (!await imageDir.exists()) {
    await imageDir.create(recursive: true);
  }
}
```

```

    await imageFile.copy(savedPath);
    return savedPath; // Returns Future<String>
  }

```

5.2 Streams — Real-Time Data

Streams emit multiple values over time. Perfect for Firestore real-time listeners.

```

// From firestore_service.dart – Firestore real-time stream
Stream<int> unseenFulfilledWishesCount() {
  return _wishesCollection(uid)
    .where('ownerSeen', isEqualTo: false)
    .snapshots() // Returns Stream<QuerySnapshot>
    .map((snapshot) => snapshot.docs.length); // Transform to Stream<int>
}

// From providers.dart – Riverpod watches streams
final unseenFulfilledCountProvider = StreamProvider<int>((ref) {
  return ref.watch(firestoreServiceProvider).unseenFulfilledWishesCount();
});

```

Future vs Stream

Future	Stream
Returns ONE value	Emits MANY values over time
<code>await</code> to get value	<code>listen()</code> or <code>StreamProvider</code>
Fetching a user profile once	Real-time wishlist updates
<code>Future<String></code>	<code>Stream<int></code>

5.3 try/catch/finally & rethrow

```
// From upload_service.dart – Full error handling pattern
try {
  final downloadUrl = await ImageStorageService.uploadImage(file, userId);
  await FirebaseFirestore.instance.collection('wishes').doc(wishId).update({ ... }
} catch (e) {
  debugPrint('Upload failed: $e');
  if (task.attempt < 3) {
    // Retry logic
  } else {
    _removeFromQueue(task);
  }
} finally {
  _isUploading = false;      // Always runs, even after errors
  _processQueue();          // Continue with next item
}

// rethrow – pass the error to the caller
catch (e) {
  debugPrint('Error: $e');
  rethrow; // Let the UI handle it
}
```

5.4 Firestore Transactions

```
// From firestore_service.dart – Atomic read-then-write
await _firestore.runTransaction((transaction) async {
  final snapshot = await transaction.get(docRef);
  final data = snapshot.data() as Map<String, dynamic>;
  final currentFulfilledBy = data['fulfilledBy'] ?? '';

  if (currentFulfilledBy.isEmpty) {
    transaction.update(docRef, {
      'fulfilledBy': fulfillerId,
      'ownerSeen': false,
    });
  }
});
```

```
// ↑ Guarantees no race conditions – if two friends click "fulfill" at the same
//    only one succeeds. The transaction retries automatically on conflict.
```

6. Design Patterns Used

6.1 copyWith Pattern (Immutability)

All our models are immutable (`final` fields). To "change" a value, we create a NEW object with the changed field.

```
// From user_model.dart
UserModel copyWith({
  String? displayName,
  List<FriendProfile>? friends,
}) {
  return UserModel(
    uid: uid, // Keep original
    displayName: displayName ?? this.displayName, // Use new OR keep original
    friends: friends ?? this.friends,
  );
}

// Usage:
final updatedUser = user.copyWith(displayName: 'New Name');
// Original user is UNCHANGED, updatedUser has the new name
```

6.2 Repository Pattern

`FirestoreService` centralizes ALL database operations. Pages never touch Firestore directly.

```
// Service layer (firestore_service.dart)
class FirestoreService {
  Future<void> createHive(HiveModel hive) { ... }
  Future<void> updateWish(WishModel wish) { ... }
  Stream<int> unseenFulfilledWishesCount() { ... }
}
```

```
// UI layer – calls service through Riverpod provider
await ref.read(firestoreServiceProvider).createHive(hive);
```

6.3 Queue Pattern (Upload Service)

```
// upload_service.dart – Background upload queue with retry logic
class UploadService extends Notifier<List<UploadTask>> {
  void addToQueue(wishId, imageFile, userId) {
    state = [...state, task]; // Add to queue
    _processQueue();          // Start processing
  }

  Future<void> _processQueue() async {
    if (_isUploading || state.isEmpty) return;
    // Upload → Update Firestore → Remove from queue → Process next
    // Retries up to 3 times on failure
  }
}
```

7. Flutter Widget System

7.1 StatelessWidget vs StatefulWidget

```
// StatelessWidget – No internal state, just builds UI
class BeehiveApp extends ConsumerWidget { // Riverpod version
  Widget build(BuildContext context, WidgetRef ref) { ... }
}

// StatefulWidget – Has mutable internal state (e.g. tab index, text controllers)
class ContactsPage extends ConsumerStatefulWidget { ... }
class _ContactsPageState extends ConsumerState<ContactsPage> {
  late TabController _tabController; // late = initialized later (in initState)

  void initState() {
    super.initState();
    _tabController = TabController(length: 2, vsync: this);
  }
}
```

```
}

void dispose() {
  _tabController.dispose(); // Clean up resources
  super.dispose();
}
}
```

7.2 Widget Lifecycle

Method	When Called	WishHive Usage
initState()	Widget created	Start animations, load contacts
didUpdateWidget()	Parent rebuilds with new data	Reset animation on wish toggle
build()	Every time setState() is called	Render the UI
dispose()	Widget removed from tree	Clean up controllers, timers

7.3 Key Flutter Widgets Used

Widget	Purpose	Where Used
Scaffold	Page structure (body + appbar + FAB + nav)	Every page
IndexedStack	Show one child at a time, keep state	home_page.dart (tab switching)
CurvedNavigationBar	Animated bottom nav bar	home_page.dart
CustomScrollView + Slivers	Advanced scrolling layouts	contacts_page.dart, home_page.dart
Hero	Shared element animation	HiveCard → ProductDetailPage
AnimatedContainer	Smooth property transitions	Wish checkbox animation

<code>FadeTransition</code>	Opacity animation	Page entry animations
<code>Badge</code>	Notification count indicator	Nav bar (friends, wishes)
<code>SpeedDial</code>	Expandable FAB menu	Create Hive / Create Wish
<code>TabBar</code> + <code>TabBarView</code>	Tabbed interface	<code>contacts_page.dart</code>

7.4 Conditional Widget Rendering

```
// if-condition inside widget tree (collection if)
Row(
  children: [
    if (isCompleted && !widget.wish.ownerSeen && widget.isOwner)
      Container(
        width: 8, height: 8,
        decoration: const BoxDecoration(color: Colors.red, shape: BoxShape.circul
      ),
    Expanded(child: Text(widget.wish.name)),
  ],
),

// Ternary in widgets
isCompleted
  ? const Icon(Icons.check, color: Colors.white)
  : null,

// Spread with if (collection for + if)
if (friends.isNotEmpty) ... [
  SliverToBoxAdapter(child: Text('Your Friends')),
  SliverList( ... ),
],
```

8. State Management (Riverpod)

8.1 Provider Types Used

```
// Provider – Simple value, no state changes
final firestoreServiceProvider = Provider<FirestoreService>((ref) {
  return FirestoreService();
});

// StreamProvider – Wraps a Stream, auto-handles loading/error/data
final currentUserStreamProvider = StreamProvider<UserModel?>((ref) {
  final uid = ref.watch(uidProvider);
  return ref.watch(firestoreServiceProvider).userStream(uid!);
});

// FutureProvider – Wraps a Future
final friendFeedProvider = FutureProvider.autoDispose<List<HiveModel>>((ref) asy
  return ref.read(firestoreServiceProvider).getFriendsFeed(user.friends);
});

// NotifierProvider – Complex state with methods
final uploadServiceProvider = NotifierProvider<UploadService, List<UploadTask>>(
```

8.2 ref.watch vs ref.read

```
// ref.watch – Rebuilds widget when value changes (use in build())
final user = ref.watch(currentUserStreamProvider).value;
final unseenCount = ref.watch(unseenFulfilledCountProvider).value ?? 0;

// ref.read – One-time read (use in callbacks, NOT in build())
await ref.read(firestoreServiceProvider).sendFriendRequest(uid);
```

8.3 AsyncValue Pattern (when)

```
// Riverpod's AsyncValue handles Loading, Error, and Data states automatically
myUserAsync.when(
  data: (myUser) {
    if (myUser == null) return Text('User not found');
    return _buildFriendsTab(myUser);
  },
  loading: () => const CircularProgressIndicator(),
```

```
error: (e, _) => Text('Error: $e'),
);
```

9. Firebase Integration

9.1 Firebase Services Used

Service	Purpose
Firebase Auth	Login (Email, Google Sign-In)
Cloud Firestore	Database (users, hives, wishes)
Firebase Storage	Image uploads
Firebase App Check	API security

9.2 Firestore Data Model

```
// Document Structure:
users/
  {uid}/
    |— email, displayName, username, photoUrl
    |— friends: [ {uid, displayName, photoUrl, email} ]
    |— friendRequestsSent: [ uid1, uid2 ]
    |— friendRequestsReceived: [ uid3 ]
    |— mutedFriends: [ uid4 ]
    |— hiddenHiveIds: [ hiveId1 ]
    |— hives/                               ← Subcollection
      {hiveId}/
        |— title, imageUrl, note, privacy
        |— itemCount, totalCost
        |— wishes/                           ← Subcollection
          {wishId}/
            |— name, imageUrl, cost, link
```

```

├─ fulfilledBy, fulfilledByName
└─ ownerSeen

```

9.3 Offline Persistence

```

// From main.dart – Works offline automatically
FirebaseFirestore.instance.settings = const Settings(
  persistenceEnabled: true,
  cacheSizeBytes: Settings.CACHE_SIZE_UNLIMITED,
);

```

10. App Architecture

10.1 Project Structure

```

lib/
├─ main.dart                # App entry point
├─ firebase_options.dart    # Auto-generated Firebase config
├─ core/
│   ├── theme/app_theme.dart # Colors, fonts, widget themes
│   ├── constants/app_constants.dart
│   └─ utils/
├─ models/                  # Data classes
│   ├── user_model.dart     # UserModel, FriendProfile
│   ├── hive_model.dart     # HiveModel (category/collection)
│   └─ wish_model.dart      # WishModel (individual item)
├─ services/                # Business logic layer
│   ├── firestore_service.dart # All CRUD operations (700+ lines)
│   ├── image_storage_service.dart
│   ├── upload_service.dart  # Background upload queue
│   ├── metadata_service.dart # URL metadata fetching
│   └─ affiliate_service.dart
├─ providers/               # Riverpod state providers
│   ├── providers.dart
│   └─ locale_provider.dart
├─ pages/                   # UI screens (15 pages)
│   └─ home_page.dart       # Main dashboard + nav

```

```

├── product_detail_page.dart # Hive detail with wishes
├── contacts_page.dart      # Friends + phone contacts
├── friend_feed_page.dart   # Friends' hives
├── create_hive_sheet.dart   # Bottom sheet for creating hive
├── create_wish_sheet.dart   # Bottom sheet for creating wish
├── ...
├── widgets/                # Reusable components
│   ├── hive_card.dart
│   ├── shimmer_loading.dart
│   └── ...
└── l10n/                   # Localization (EN, FR, HI, TE)

```

10.2 Data Flow

// REQUEST FLOW:

UI (Page) → ref.read(provider) → Service → Firestore

// REALTIME DATA FLOW:

Firestore → Stream → StreamProvider → ref.watch() → UI auto-rebuilds

11. Complete Feature List

#	Feature	Description
1	 Authentication	Email/password + Google Sign-In, profile completion
2	 Hives (Categories)	Create, edit, delete wishlists with images and privacy settings
3	 Wishes (Items)	Add wishes with name, image, price, link, quantity, notes
4	 Friends System	Send/accept/reject friend requests, mute friends
5	 Phone Contacts	View contacts, invite to app via SMS/WhatsApp
6	 Wish Fulfillment	Friends can mark wishes as "I'll get this for you"

7	 Notification Dots	Badge on nav icon + dot on wish when friend fulfills
8	 Share Intent	Receive shared links from Amazon/Blinkit/etc → auto-create wish
9	 Image Management	Local storage, Firebase upload, compression, full-screen view
10	 Privacy Controls	Public/Private/Friends/Specific viewer access
11	 Hidden Hives	Hide specific hives from feed
12	 Multi-Language	English, French, Hindi, Telugu
13	 Animations	Hero transitions, shimmer loading, fade transitions
14	 Offline Support	Firestore persistence, background upload queue
15	 Marketplace	Product recommendations (planned)

12. Future Improvements

Onboarding Tutorial

Step-by-step coach marks guiding new users through creating their first hive and wish. Uses `tutorial_coach_mark` package with a `hasCompletedTutorial` field in Firestore.

Push Notifications (FCM)

Real push notifications for wish fulfillments, friend requests, and friend activity. Uses Firebase Cloud Messaging with Cloud Functions as triggers.

AI-Powered Features

- **Smart Price Tracking** — Track prices across e-commerce sites and notify on drops
- **Auto-Categorization** — AI suggests which hive a new wish belongs to
- **Gift Recommendations** — Suggest gifts based on friend's wishlist patterns

Analytics Dashboard

Show users stats like: wishes fulfilled, total savings, most active friends, spending patterns per category.

Deep Linking

Share a hive link → recipient opens the app directly to that hive. Uses Firebase Dynamic Links or App Links.

Theme Customization

Dark mode, custom accent colors, and seasonal themes (Diwali, Christmas).

Testing

- Unit tests for all services (FirestoreService, UploadService)
- Widget tests for complex widgets (WishTile, HiveCard)

- Integration tests for auth flow and CRUD

⚡ Performance

- Pagination for large wishlists (currently loads all at once)
- Image caching with aggressive TTL
- Lazy loading for friend feed

WishHive — Built with Flutter & Firebase

Generated on February 16, 2026 | For interview preparation & learning reference

